Procedural Input Verification for Clinical Tool Calling: An Empirical Study of Interwhen Verifier-Guided Reasoning with LLM-Extracted References

Generative AI · Medical informatics · Intermediate-step verification

Jack Yang, Jaeyoon Wang, Aly Moosa

Bill & Melinda Gates Foundation

May 28, 2026

Abstract.

[ABSTRACT TBD — update after primary study rerun.]

Verifier-guided reasoning (interwhen-style) intercepts an LLM's intermediate reasoning steps and asks an external verifier to approve them before they take effect. We evaluate this mechanism in a clinical decision-support setting — specifically, *procedural input verification* of tool-call payloads against patient facts extracted from case vignettes — using medical_calculator_eval ($N = 1066$) and Qwen3-30B-A3B-Thinking-2507 as the primary model. The verifier compares each (field, value) pair the model proposes to dispatch to an MCP medical calculator against a reference set of facts extracted from the case by Sonnet 4.6.

We study a single experimental factor, the architecture of the input-verification pipeline, across four primary conditions presented at their best-practice configuration (schema-gated verifier comparison, query-style intervention prompts, prompt-only calibrated abstention). The four conditions span three orthogonal axes: extraction placement (upfront full-schema vs reactive per-tool-call), output format (bare values vs citation-anchored pairs), and sampling ($k = 1$ vs $k = 3$ majority vote). Six paired McNemar contrasts are pre-registered at family $\alpha = 0.05$ (Bonferroni-corrected per-test $\alpha = 0.00833$).

[Headline results TBD] — to be filled after the primary study rerun. Expected structure: (1) whether any extractor pipeline improves accuracy over the forced-tool baseline B′ at the pre-registered significance level; (2) which architectural axis (placement, format, sampling) carries the effect, if any; (3) cost-accuracy tradeoffs across the four pipelines.

The best-practice configuration was motivated by pilot diagnostics reported in section 3.5; the schema gate and query-style intervention address, by construction, two failure modes those pilots exposed.

Key Findings

[KEY FINDINGS TBD — update after primary study rerun.]

- **[TBD]** Does verifier-guided extraction at best-practice configuration (schema-gated, query-intervention, prompt-only abstention) improve accuracy over the forced-tool baseline B′? Report McNemar p vs B′ for upfront, reactive, reactive+citations, reactive+k-shot.
- **[TBD]** Does extraction placement (upfront vs reactive) carry the effect? Within-architecture contrast at fixed output format and sampling.
- **[TBD]** Do mechanism arms close the gap? Citation grounding (reactive+citations vs reactive) and k-shot voting (reactive+k-shot vs reactive).
- **[TBD]** Cost-accuracy tradeoff: which architecture is Pareto-efficient on total tokens, on API tokens, and on median wall-clock latency.

Keywords: clinical AI safety, procedural input verification, verification, interwhen, large language models, reasoning trace, fact extraction, medical calculators, KARMA, Model Context Protocol, agent evaluation

1 Introduction

1.1 Motivation: Wrong Inputs Defeat Correct Calculators

Clinical decision-support is increasingly delivered by large language models (LLMs) that call deterministic backend tools — risk calculators, dosing engines, eligibility checks — via a structured tool-calling interface. The deterministic backend is the easy part: a correctly implemented CHA_2DS_2-VASc calculator returns the right score on any input. The hard part is the boundary where the LLM hands the calculator its inputs. A correct calculator called with a transposed creatinine value, the wrong patient sex, or a miscoded comorbidity will return a plausible-but-wrong answer that no amount of downstream verification of the textual response can detect.

Concretely, in the medical_calculator_eval benchmark used in this study, many vignettes admit only one "correct" calculator and one "correct" input mapping. A model that names the right calculator but misreads the patient's sex from the case will receive a confidently wrong score and present it as the answer. The failure mode is structural: it lives at the LLM-to-tool input boundary, not in the calculator and not in the final textual summary.

1.2 Scope: Procedural Input Verification

We study *procedural input verification* of tool calls: an intermediate intervention that inspects the JSON payload the LLM sends to a deterministic clinical calculator *before* the calculator runs. This is distinct from output verification (re-checking the final textual answer), end-to-end outcome verification (was the diagnosis right?), and retrieval grounding (was the cited fact in the source?). The verification fires at one specific point in the agent loop: between the model emitting a `<tool_call>` block and the MCP tool being invoked. The verifier compares each (field, value) pair the model proposes to dispatch against an independently extracted reference set of patient facts. If the case (and the extractor) say sex = female and

the model proposes sex = male, the call is flagged and the model is asked to revise before the calculator runs.

This is the scope of every claim in this paper. We do not study other verification surfaces, and our results say nothing about them directly.

1.3 The Verification Hypothesis

We study a single experimental factor: the architecture of the input-verification pipeline. Four primary hypotheses are pre-registered, one per architectural axis (section 3.8):

H1 (placement). Best-practice procedural input verification with upfront full-schema extraction improves accuracy over the forced-tool baseline B′.

H2 (placement). Best-practice procedural input verification with reactive per-tool-call extraction improves accuracy over B′.

H3 (output format). Adding citation grounding to the reactive pipeline (`value` + verbatim source span) improves accuracy over the bare-value reactive baseline.

H4 (sampling). Replacing single-sample reactive extraction with $k = 3$ majority voting improves accuracy over the single-sample reactive baseline.

"Best-practice" here means schema-gated verifier comparison, query-style intervention prompts, and prompt-only calibrated abstention — the three hygiene properties baked into all primary study conditions (section 3.4). These properties are not factors under test; they were motivated by pilot diagnostics (section 3.5) and are held constant across H1–H4.

1.4 Contributions and Roadmap

This paper contributes the following. (i) A nine-condition evaluation of procedural input verification on a clinical tool-calling benchmark under a best-practice pre-registered design that varies only the extractor pipeline across the four primary study conditions. (ii) Six pre-registered McNemar contrasts (Bonferroni $\alpha = 0.00833$) testing four hypotheses about extractor architecture: placement (upfront vs reactive), output format (bare vs citation-anchored), and sampling ($k = 1$ vs $k = 3$ majority vote). (iii) A pilot-diagnostic methodology section (section 3.5) that documents the un-gated and correction-style variants that motivated the best-practice hygiene baked into the primary study. (iv) A uniform per-condition logging schema and analysis surface that makes the four extractor pipelines directly comparable on accuracy, flag composition, and cost.

We do *not* claim that interwhen "does not work." Our results adapt the mechanism to a setting where the verifier's reference must be an independent fact set extracted from the case — a ground-truth reference layer that interwhen's trace-state extraction does not have to produce — and the binding constraint, if one remains under best-practice configuration, is jointly that reference pipeline and the verifier's comparison surface.

The rest of the paper is organized as follows. Section 2.1 reviews the interwhen mechanism and the clinical-tool-calling setting. Section 3.1 describes the apparatus, the nine conditions, the pre-registered configuration, and the statistical analysis. Section 4.1 reports the apparatus gate and the six pre-registered contrasts;

sections 4.3 and 4.4 develop the within-architecture comparisons and the post-hygiene flag composition. Section 5.1 synthesizes what the results show and the implications for deploying intermediate verification in clinical AI.

2 Background

2.1 Verifier-Guided Reasoning (Interwhen)

Interwhen [2] is a verifier-guided reasoning framework in which a monitor periodically polls an LLM's reasoning trace, forks the model to extract verifiable state from the partial trace, and runs verifiers on that state asynchronously — injecting feedback and re-prompting only when a violation is detected. The verifiers are code-based predicates over partial traces and can be synthesized automatically from natural-language policy documents, including provably correct variants in Lean and z3. Interwhen is evaluated across both non-agentic reasoning (math, logic) and agentic tool-calling benchmarks (e.g., τ^2-bench). Crucially for our adaptation, the state the verifier consumes is itself recovered by an LLM extraction step over the trace, and interwhen explicitly analyzes the framework's robustness to errors in that extraction.

We adapt the mechanism rather than re-implement it, preserving interwhen's verify/fix control flow exactly — on flag, inject feedback and re-prompt; on clean, dispatch the tool and continue. Three things change. First, instead of polling the trace at intervals, we fix the intercept at a single deterministic commit point: the moment the primary model emits a `<tool_call>` block, whose parsed JSON is the state under check. Second, the LLM extraction reads the patient *case* — the task input — rather than the model's own reasoning trace; interwhen recovers the model's asserted state from the trace and checks it against a policy, whereas we recover an independent reference from the case and check the model's proposed inputs against it. Third, in place of a policy-synthesized verifier we use a hand-built deterministic field-by-field comparator. The transport also differs: we use vLLM's text-completions endpoint with a custom synchronous loop driver rather than the streaming variant, because the primary model is served behind a standard completions API.

2.2 Clinical Tool-Calling and KARMA

KARMA [4] is a structured evaluation framework for medical AI published by EkaCare. We use its publicly released medical_calculator_eval benchmark ($N = 1066$ vignettes spanning 24 calculator categories, from cardiology to obstetrics) and the associated MedAI Model Context Protocol (MCP) toolset [5].

The MCP toolset exposes six meta-tools, of which three are central to this study: *medical_calculator_list* enumerates available calculators; *medical_calculator_input* returns the JSON Schema for a chosen calculator's input; and *medical_calculator_output* dispatches the calculator with an `input_data` payload. Each calculator carries a typed JSON Schema with enums, ranges, and field descriptions. The union of input field names across all calculators yields a vocabulary of ~500 unique fields. This union is the reference vocabulary the verifier and extractor share, by construction — both consume the schemas EkaCare ships, so neither can refer to a field the other does not know about.

2.3 Verification Surfaces: Where to Intervene

LLM-driven clinical AI admits intervention at several surfaces, which are often conflated in casual discussion. We distinguish:

Input verification (this paper). The verifier inspects the model's proposed tool-call arguments *before* dispatch. Catches wrong inputs at the boundary where they would otherwise propagate into a deterministic backend. Fires at intermediate commit points; firing frequency scales with tool-call rate.

Output verification. The verifier reads the model's final textual answer and may request revision. Cannot catch a wrong tool input that produced a confidently-stated wrong answer unless the final text exposes the inconsistency. Condition D in our study is the closest output-style comparator.

Grounding / retrieval-augmented verification. The reference is retrieved text. A different problem class — the question becomes "is the claim supported by retrieved evidence?" rather than "is the proposed input consistent with the case?" Not evaluated here.

Self-verification. The model is asked, in-prompt, to check itself. No external reference. Condition C is our control for "is any external machinery necessary?"

This paper concerns the first surface. Every claim about "the verifier" refers to procedural input verification specifically.

Procedural input verification is a *special case* of reasoning-trace verification: the model's tool-call emission is one step in its trace, and we intercept at that step. What distinguishes our setting is where the verifier's reference comes from and where the commit point sits. Interwhen recovers verifiable state by polling the trace at intervals; in a clinical agent most of that trace is unstructured prose, and the reference the verifier needs — the patient's actual values — is not in the trace at all but in the case text. The model's typed tool-call payload is the first point in the trace where a machine-checkable structured commit exists, and the calculator's input schema defines exactly what fields constitute that commit. A flag therefore localizes to a specific (calculator, field, value) tuple — auditable in the way clinical safety reviews require. Second, existing clinical decision-support pipelines already *have* these tool boundaries (risk calculators, formularies, eligibility checkers, FHIR queries), so input verification inserts at a place the deployment architecture already exposes, rather than requiring the workflow to be restructured around inline reasoning checks.

2.4 The Reference-Extraction Problem

A subtlety in the clinical adaptation deserves explicit attention: *the reference itself must come from somewhere.* Interwhen already relies on an LLM to extract the verifier's input variables, but it extracts them from the model's own reasoning trace — the state the verifier checks is the model's asserted state, scored against a policy. In our setting the verifier instead needs the patient's true values, which live only in the natural-language case. Before the verifier can compare anything, the case must be parsed into field–value pairs by an LLM, and that extracted record is then treated as the reference against which the model's proposed inputs are judged. The model-based extraction step is therefore not new to interwhen; what is new is that it now stands between the case and the verifier as the source of ground truth, which is where the verifier's safety guarantee can weaken.

We use an external LLM of a different model family than the primary to produce the reference (specific model, version, and serving configuration in section 3.1). The cost of this design is that the extractor is

itself an LLM, with its own distribution of errors. Our central hypothesis (section 1.3) is that those errors — not the verifier mechanism — are the binding constraint on achievable accuracy in this setting; whether the data bear that out is what the study tests.

2.5 Which Failure Classes Input Verification Can Reach

EkaCare's own error analysis of medical_calculator_eval [3] identifies two failure classes that persist even with tool access. The first is *wrong calculator selected* — the model picks a semantically adjacent but distinct tool (e.g., a generic BMI calculator where a sex-specific variant is required). The second, affecting 12–13% of samples across the three models they evaluated, is *right calculator, wrong inputs*, which decomposes into (a) enum misselection (mapping "sedentary" to "very_active" in a TDEE activity field) and (b) unit-conversion overreach, where the model receives a correct calculator result and then rewrites it into a different unit, introducing an error the calculator did not make.

Procedural input verification is scoped to exactly one of these sub-classes. It fires before dispatch on the tool-call payload, so it can catch enum misselection and analogous wrong-input errors at the boundary — sub-class (a) above. It *cannot* reach sub-class (b): unit-conversion overreach occurs after the calculator returns, on the output surface, where only a post-hoc check such as Condition D could intervene. Wrong-calculator selection is likewise out of scope — the verifier checks field values for the calculator the model has already chosen, not the choice of calculator itself. The addressable ceiling for the primary study conditions is therefore the enum/wrong-input portion of the 12–13% band, not the full band; we report the split where the logs permit and calibrate expected effect sizes accordingly. Condition D is the only arm positioned to reach the unit-overreach sub-class, and it targets a different failure than the B′+E conditions rather than a more thorough version of the same one.

3 Methods

3.1 Benchmark and Primary Model

The benchmark is medical_calculator_eval (Hugging Face dataset *ekacare/medical_calculator_eval*, test split, $N = 1066$ vignettes), released by EkaCare as part of the KARMA evaluation framework [4]. We use the public release without modification.

Each item is a single-turn task: a case-narrative vignette in natural-language prose, a target calculator, an *expected numerical value* for a designated primary output field, and a per-item tolerance. There is no conversation — the model receives the vignette once and produces a single response. Each item also carries a per-row *confinement instruction* — a prompt suffix that instructs the model to reply with a single JSON object containing the primary field — which the adapter appends to the vignette before generation. Scoring is binary: the harness extracts a numerical prediction from the model's reply (preferring fenced JSON blocks, falling back to the last number in the text), and the item is marked correct iff $|\text{predicted} - \text{expected}| \leq \text{tolerance}$. Parse failures and missing-field responses score as wrong. All accuracy numbers in this paper are this strict tolerance-based binary correctness, not partial-credit scores. Critically, the input is free-text prose rather than structured fields — the reason an LLM extractor is required to populate the verifier's reference at all. Structured-input benchmarks (e.g., EHR-keyed inputs) would remove the extraction bottleneck this paper diagnoses. Curation details (vignette authorship, validation, calculator coverage) are documented in the KARMA release. The primary model under evaluation is Qwen3-30B-

A3B-Thinking-2507 [8], served via vLLM [6] on the text-completions endpoint with temperature 0.0, top-p 1.0, max generation length 4096 tokens, and a tool-calling loop bounded at 10 turns. The primary model's sampling is deterministic by construction (temperature zero, frozen MCP schema dump). The extractor's sampling varies by condition and is documented in section 3.4.

The fact extractor used in all extractor-enabled conditions is Claude Sonnet 4.6 [1], called with a single pinned model snapshot for the duration of each run. The post-hoc verifier used in exploratory condition D is also Sonnet 4.6. We chose Sonnet rather than a sibling of Qwen3 to ensure the verifier's reference is not produced by the same model family as the primary, which would defeat the purpose of an external check.

3.2 Compute and Serving Infrastructure

Qwen3 is served by a single vLLM process on an Azure NC40ads-H100-v5 instance (one NVIDIA H100 80 GB GPU), running on Databricks Runtime 17.3 (GPU/ML build, Scala 2.13). Inference uses the text-completions endpoint, not chat completions, so the custom tool-calling loop driver can intercept generation at `</tool_call>` stop tokens. Per-condition runs are parallelized across vignettes via a process pool; the worker count ranges from 32 to 128 depending on per-vignette Sonnet pressure (higher pressure → lower concurrency to respect API rate limits). Sonnet calls are dispatched against Anthropic's public API; no model weights other than Qwen3 run locally. Per-condition GPU-hours and estimated cloud costs are recorded in each result bundle's provenance file.

Because the worker count varies across conditions, per-vignette wall-clock from the accuracy run is not comparable across conditions (contention differs by load). Accuracy and token counts are load-independent and are taken from the full run; *latency* is measured separately in a fixed-concurrency pass that re-runs a 50-vignette subset of every condition at a single worker, yielding a consistent single-request (deployment-style) latency. All latency numbers in section 4.6 come from that pass.

3.3 Conditions: Design and Purpose

The study is organized around a single experimental factor: the *architecture of the input-verification pipeline*. Nine conditions are partitioned into three groups (table 1): three *anchors* that bound the room for verification to improve, four *primary study* conditions that vary the extractor architecture under a fixed best-practice verifier, and two *exploratory* conditions that test alternative verifier mechanisms (in-prompt self-verify and output-surface post-hoc check).

Each primary study condition is presented at its best-practice configuration: schema-gated verifier comparison, query-style intervention prompts, and prompt-only calibrated abstention. These three hygiene properties are identical across all primary study conditions and are documented in section 3.4; they are not factors under test. The naive un-gated and correction-style variants used in earlier pilot runs are documented as design motivation in section 3.5.

3.3.1 Capability Anchors (No Extractor)

A disables tools entirely and uses no system prompt; it is the capability floor, reporting what the primary model can answer from internal knowledge alone.

B enables MCP tools with no system prompt; it is the pre-registered baseline used for the apparatus gate (section 3.9). If Sonnet's accuracy on B falls within ±3 pp of the published 81.9% reference, the harness is

treated as drift-free.

B′ enables tools and adds a system prompt that instructs the model to invoke a calculator rather than answering from prior knowledge (verbatim prompt in listing 1). B′ isolates the effect of forced tool use independent of any verifier and serves as the comparator for the primary study conditions.

3.3.2 Primary Study: Extractor Architecture

Four conditions share an identical verifier and intervention design; they differ only in the extractor pipeline. All four use the B′ system prompt, the schema-gated deterministic verifier (section 3.6), query-style intervention prompts, and prompt-only abstention.

B′+E (upfront). One Sonnet call per vignette, at run start, populating the union of all calculator schemas (~500 fields). The extracted record is consulted for every subsequent tool-call interception.

B′+E (reactive). One Sonnet call per *medical_calculator_output* tool call, scoped to the fields the active calculator declares. The extractor sees only the field names the calculator requests and the case text; it never sees the values the primary model proposed.

B′+E (reactive + citations). Reactive extraction with the extractor returning, for each populated field, the exact source span in the vignette that supports the value. The span must match a substring of the vignette verbatim; if validation fails, the field is treated as null (abstention).

B′+E (reactive + k-shot). Reactive extraction sampled $k = 3$ times per tool call at temperature 0.7 with three independent seeds. The three samples are reduced per field by majority vote; three-way disagreement yields null (abstention).

The architectural axis isolated by each pairwise contrast is given in table 1 and the contrast logic in section 3.8.

3.3.3 Exploratory: Other Verifier Mechanisms

Two conditions test verifier mechanisms outside the extractor–verifier architecture family. They are not in the primary contrast family and inherit their numbers from earlier pilot runs under fixed apparatus.

C (prompt-only self-verify) instructs the model in-prompt to verify each input value against the case before calling a tool (prompt in listing 2). No external extractor, no separate verifier.

D (post-hoc Sonnet verifier) sends `(case, candidate answer)` to a Sonnet reviewer after the primary model produces a final answer (prompt in listing 3). On flag, the model is re-prompted once with the reviewer's one-sentence issue. D is the closest analog to "check the final answer" verification approaches.

3.4 Pre-Registered Configuration

The four primary study conditions are run under identical hygiene and sampling configuration; the only inter-condition variation is the extractor pipeline. All values below are pre-registered and locked prior to the study run; no tuning to results is permitted.

Table 1: Nine conditions. Primary study conditions share identical hygiene (schema-gated verifier, query intervention, prompt-only abstention); they vary only in the extractor pipeline.

Condition	Group	Tools	Sys. prompt	Verifier	Extractor pipeline
A	anchor	no	none	none	–
B (baseline)	anchor	yes	none	none	–
B′	anchor	yes	force-tool	none	–
B′+E (upfront)	primary	yes	force-tool	deterministic	upfront full-schema
B′+E (reactive)	primary	yes	force-tool	deterministic	reactive per-call
B′+E (reactive + citations)	primary	yes	force-tool	deterministic	reactive + source spans
B′+E (reactive + k-shot)	primary	yes	force-tool	deterministic	reactive + k=3 vote
C	exploratory	yes	self-verify	prompt-only	–
D	exploratory	yes	none	post-hoc Sonnet	–

3.4.1 Baked-In Hygiene

Schema-gated verifier comparison. The verifier compares *arguments["input_data"]* against the extracted record only on fields declared as `required` in the active calculator's JSON Schema. Fields the extractor populated but the calculator does not require are ignored at verify time. This suppresses flags on fields that cannot affect the calculator's output and was motivated by the pilot diagnostic in section 3.5 showing 58.6% of legacy-condition flags fired on non-required fields.

Query-style intervention. On flag, the verifier injects a prompt that presents the disagreement as a question to be resolved against the vignette, not as an assertion of model error (verbatim template in appendix A.3). The format names the disagreeing field, the model's proposed value, and the extractor's value, and asks the model to consult the case to determine which is supported.

Prompt-only calibrated abstention. The extractor prompt instructs Sonnet to return `null` for any field whose value the case does not clearly state or imply. The verifier's existing no-flag rule on missing extractor values (section 3.6) then applies. No confidence thresholding; no post-hoc adjustment.

3.4.2 Locked Run-Time Parameters

Table 2 fixes the parameters that govern sampling, re-prompting, and failure handling. These are identical across all primary study conditions; the only inter-condition variation lives in the extractor pipeline row.

3.4.3 Pilot-Batch Sanity Checks

Before committing to the full $N = 1066$ run, a 30-vignette pilot batch verifies five properties of the implementation: (1) k-shot samples diverge string-wise across the three seeds; (2) the Qwen3 input for a given vignette in conditions B and B′ is bit-identical (catches preprocessing drift); (3) the re-prompt cap halts the loop when reached; (4) the abstention prompt produces `null` on a vignette with intentionally missing information; (5) the citation validator rejects a paraphrased source span. Failures on the pilot batch block the full run.

Table 2: Pre-registered run-time parameters for the primary study. Locked prior to the study run.

Parameter	Value
Sonnet snapshot	pinned per-run; logged in result bundle
Sonnet extraction temperature	0.7
Sonnet extraction k samples	1 (primary 3 cond.); 3 (k-shot cond.)
Sonnet seed strategy	fresh per call; no prompt caching
Qwen3 temperature	0.0
Tool-call loop turn cap	10
Verifier re-prompt cap (total)	2 per vignette
Citation validity rule	verbatim substring of vignette
Citation validation failure	treat field as null (abstain)
k-shot voting rule	per-field majority of 3
k-shot no-majority behavior	treat field as null (abstain)
Failure handling (incomplete vignettes)	drop from analysis; report dropped-N per condition

3.5 Pilot Diagnostics (Design Motivation)

The hygiene properties baked into the primary study were not arbitrary defaults. Two pilot variants of B′+E (un-gated comparison surface, correction-style intervention) ran prior to the primary study and produced two findings that motivated the design.

First, under the un-gated upfront variant, the verifier compared every field the extractor populated against the model's tool call, regardless of whether the active calculator required the field. Of 1,062 total violations recorded across $N = 1066, 622$ (58.6%) fired on fields the active calculator's JSON Schema does not declare as required; 181 of those 622 were on the `sex` field alone. The schema-gated comparison rule (section 3.4) eliminates this class of flag by construction.

Second, the correction-style intervention prompt ("Your planned tool call has an input that does not match the patient case; re-read the case and call the tool again with corrected inputs"') re-prompted the model with an assertion that the verifier was correct. The query-style template (appendix A.3) presents the same information without that assertion, which lets the model adjudicate against the vignette rather than defer to the verifier's reference.

These pilot results are reported in the result bundle for reproducibility but are *not* conditions in the primary study. The primary study runs the best-practice versions only.

3.6 The Schema-Gated Deterministic Verifier

A note on naming. We call the verifier *deterministic*, not *semantic*. Its comparison logic is purely structural: exact match on enums, numeric tolerance on numbers, canonical-form match on booleans. It does not reason about paraphrase, implication, equivalent encodings, or context. The schema gate is the only non-trivial filter and is purely structural as well: it consults the active calculator's JSON Schema and restricts comparison to the fields the calculator declares as `required`.

Figure 1 shows the data flow. For each `<tool_call>` emitted by Qwen3, the verifier (a) parses *arguments["input_data"]* into (field, value) pairs; (b) restricts the comparison set to fields the active calculator's schema declares as required (*schema gate*); (c) for each remaining pair, looks up the extractor's value for

that field; (d) if the extractor has no value (abstention or missing), *skips* the field (no flag); (e) if both have values, applies the deterministic comparison and flags on mismatch.

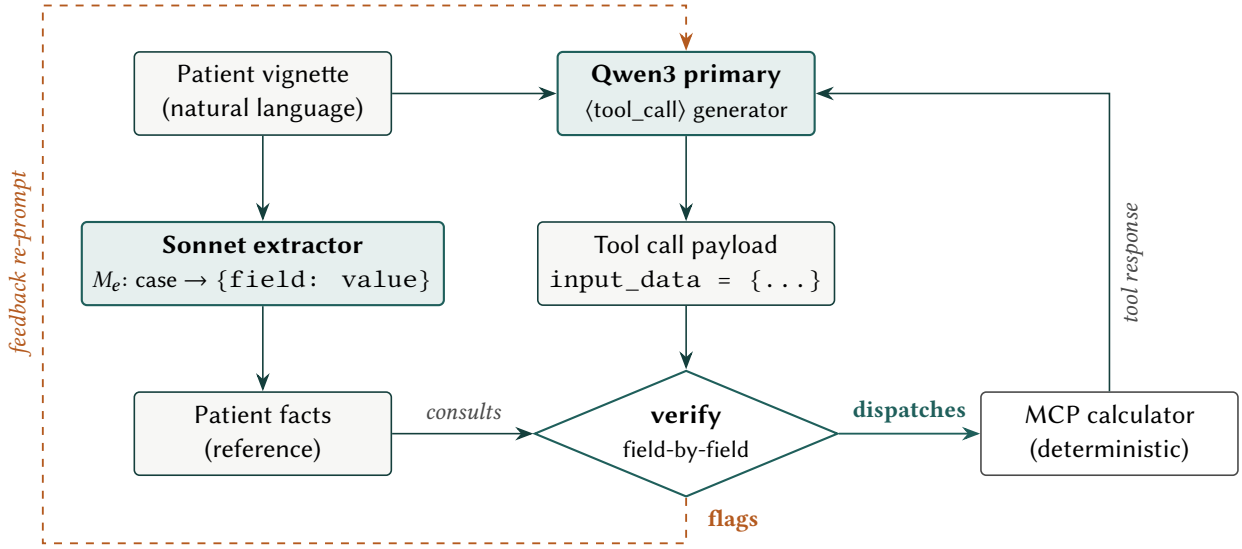

Figure 1: Apparatus overview. Right: a clean tool call is dispatched to MCP. Dashed left: a verifier flag re-prompts the primary.

On flag, the verifier injects the query-style intervention template (appendix A.3) into the model's prompt — naming the disagreeing fields, the value the model proposed, and the value the extractor found — and re-prompts. The MCP call is *not* dispatched. On clean, the call is dispatched and the tool response is injected back into the prompt; the loop continues. On malformed JSON, the verifier requests a correction without comparison. Re-prompts per vignette are capped at the value in table 2.

Three properties of this design are critical. First, the verifier is *blind* to whether the extractor was right — it only knows whether the model and the extractor disagree on a required field both populated; the extractor's value is treated as the reference by construction. Second, the verifier's "no value → no flag" rule conservatively suppresses false positives when the extractor abstains. Third, the schema gate guarantees that flags on fields the calculator's output does not depend on cannot occur, so the verifier's harm cannot be attributed to spurious flags on inactive fields.

3.7 The Fact Extractor

The extractor's vocabulary is generated deterministically from the MCP per-calculator JSON Schemas via the schema dump. Because both the extractor and the verifier consume the same schemas, the verifier *cannot ask about a field the extractor cannot in principle emit*, and the extractor cannot emit a field the MCP backend does not recognize. This vocabulary alignment is a structural property of the design, not a runtime invariant we have to police.

Four extractor pipelines are evaluated. All four share the same prompt header (listing 4), abstention rule, and sampling temperature (0.7). They differ on three orthogonal axes:

- **Placement.** Where extraction happens relative to tool calls: upfront (full-schema, once per vignette) or reactive (scoped to the active calculator, once per tool call).

- **Output format.** Whether each extracted field is a bare value or a `(value, source_span)` pair anchored in the vignette text.
- **Sampling.** Whether each field's value comes from a single sample ($k = 1$) or a majority vote over $k = 3$ independent samples.

Each primary study condition is a single point in this space:

- B′+E (upfront): upfront full-schema, bare values, $k = 1$.
- B′+E (reactive): reactive scoped, bare values, $k = 1$.
- B′+E (reactive + citations): reactive scoped, citation pairs, $k = 1$.
- B′+E (reactive + k-shot): reactive scoped, bare values, $k = 3$ vote.

The mechanism arms (citations, k-shot) anchor on reactive rather than upfront for two reasons. First, $k = 3$ over a ~50K-token upfront prompt is not a credible deployment configuration. Second, each pairwise contrast against the reactive baseline isolates a single axis change (section 3.8), preserving orthogonality.

3.7.1 Upfront Pipeline

One Sonnet call per vignette, at run start, covering the full ~500-field union schema. The result is fixed for the duration of the vignette and consulted on every tool-call interception. Across all ~70 calculators in MedAI MCP, the union of property names exceeds 500 unique fields; the schema for any single calculator typically declares 3–30. One representative calculator's JSON Schema is reproduced verbatim in appendix A.4.

3.7.2 Reactive Pipeline

One focused Sonnet call per *medical_calculator_output* tool call. At each such call, the adapter parses the keys of the model's proposed `input_data`, builds a focused Sonnet prompt restricted to those fields, and runs extraction blind to the model's proposed values (the extractor sees field *names* and the case text, never the values the model proposed). The result mutates the monitor's facts for the duration of that tool call's verification. Per-vignette cost scales with the number of `_output` calls; each call is small (typically 3–30 fields).

3.7.3 Reactive + Citations Pipeline

Identical to the reactive pipeline, except each populated field is returned as `{value, source_span}` where `source_span` is a verbatim substring of the vignette text. A post-extraction validator confirms the substring property; if validation fails, the field is treated as null (abstention). Validation rejects paraphrase, normalization, and reformatting; only byte-identical spans are accepted. The verifier consults the `value` field for comparison but logs both for analysis.

3.7.4 Reactive + k-Shot Pipeline

Identical to the reactive pipeline, except each focused Sonnet call is dispatched $k = 3$ times with three independent seeds and identical prompt at temperature 0.7. Samples are reduced per field by majority

vote: a value present in ≥ 2 of 3 samples wins; three-way disagreement yields null (abstention). The three samples are dispatched sequentially, so the per-call extraction latency of this pipeline is approximately $3\times$ that of the single-sample reactive pipeline; the cost–latency reporting (section 4.6) reflects this.

The only architectural differences across the four pipelines are *when* extraction happens, *what* the extractor returns, and *how many independent samples* are taken. The verifier, the monitor, the MCP transport, the loop driver, and the response schema are byte-identical across all four.

3.8 Pre-Registered Statistical Analysis

The primary contrast family targets the experimental factor (extractor architecture). Six paired comparisons are pre-registered; each isolates a single design axis:

1. B vs A — does tool access help?
2. B′ vs B — does forced tool use help, without verifier?
3. B′+E (upfront) vs B′ — does upfront verifier-guided extraction help over forced-tool baseline?
4. B′+E (reactive) vs B′ — does reactive verifier-guided extraction help over forced-tool baseline?
5. B′+E (reactive + citations) vs B′+E (reactive) — does citation grounding improve the reactive pipeline?
6. B′+E (reactive + k-shot) vs B′+E (reactive) — does k-shot voting improve the reactive pipeline?

With family $\alpha = 0.05$ and Bonferroni correction over $n = 6$ tests, the per-test threshold is $\alpha = 0.00833$. The outcome is paired binary correctness per vignette; the test is McNemar's exact test [7] on the discordant pairs. Per-condition accuracies are reported with Wilson [9] 95% confidence intervals. Effect sizes (Δ accuracy in percentage points) are reported alongside p-values.

A secondary contrast — B′+E (upfront) vs B′+E (reactive), isolating placement holding all other axes fixed — is reported as a within-architecture comparison; it is included in the same Bonferroni family if pre-registered as the seventh contrast, or as exploratory if not. The exploratory comparators C and D are not in the primary family and are not Bonferroni-corrected.

3.9 Apparatus Gate

Before accepting primary results, we replicated Sonnet on Condition B and required the resulting accuracy to fall within $[0.789, 0.849]$ — ±3 pp of EkaCare's published 81.9% reference for Claude Sonnet 4.6 with tool access on medical_calculator_eval [3]. The gate threshold was specified in advance in the locked pre-registration config (recorded in each result bundle's manifest, appendix A.7) and is not adjustable post-hoc. Its purpose is drift detection on the harness, dataset, prompt rendering, and MCP transport. It is not a validation of the primary Qwen3 results — we assume the harness behaves identically for the primary model conditional on the gate.

4 Results

4.1 Apparatus Gate

[TBD — fill from apparatus-gate JSON in result bundle] Sonnet on Condition B reached XX.X% accuracy ($N = 1066$), **[within / outside]** the pre-registered band defined in section 3.9. The gate **[passed / failed]** (Δ = XX.X pp). Primary contrasts below are interpretable only because this gate held.

4.2 Primary Comparisons (Pre-Registered)

Table 4 reports per-condition accuracy with Wilson 95% confidence intervals; table 5 reports the six pre-registered McNemar contrasts at Bonferroni $\alpha = 0.00833$.

[FIGURE TBD]
Forest plot: per-condition accuracy with Wilson 95% CIs ($N = 1066$). Order: A, B, B′, B′+E (upfront), B′+E (reactive), B′+E (reactive + citations), B′+E (reactive + k-shot), C, D. Color encodes group (anchor / primary / exploratory).

Figure 2: Per-condition accuracy with Wilson 95% confidence intervals ($N = 1066$). Color encodes condition group.

[TBD — headline paragraph.] Replace with the headline result once numbers are in. Recommended structure: (1) does any primary condition beat B′ after Bonferroni? (2) does extraction placement matter (upfront vs reactive)? (3) does any mechanism arm (citations, k-shot) improve over the reactive baseline?

What this does not show. A null primary result would mean that, under best-practice configuration, none of the tested extractor pipelines moved accuracy above B′ at the pre-registered significance level — not that "verifier-guided extraction cannot help" in principle.

4.3 Extractor Architecture: Placement, Output Format, Sampling

This subsection reports the within-architecture contrasts among the four primary study conditions. Each pairwise comparison isolates a single axis (section 3.7).

Placement (upfront vs reactive). *[TBD]* Compares B′+E (upfront) against B′+E (reactive) holding output format and sampling fixed at bare/$k = 1$. Report McNemar p, discordant counts, and Δ accuracy.

Output format (bare vs citation). *[TBD]* Compares B′+E (reactive) against B′+E (reactive + citations) holding placement and sampling fixed.

Sampling (k=1 vs k=3). *[TBD]* Compares B′+E (reactive) against B′+E (reactive + k-shot) holding placement and output format fixed.

4.4 Flag Composition

[FIGURE TBD]
Top-10 flagged fields under each primary study condition. Compare against the pilot un-gated upfront diagnostic (section 3.5).

Figure 3: Top-10 flagged fields under each primary study extractor pipeline.

[TBD] Report the post-rerun flag composition under schema-gating. By construction, flags on non-required fields are zero. Report the residual flag distribution and whether `sex` or any other field still dominates among required-field flags.

4.5 Per-Category and Stratified Effects

[FIGURE TBD]
Accuracy by calculator category × condition for the 9 conditions.

Figure 4: Accuracy by calculator category × condition.

[TBD] Report whether any primary condition's effect is concentrated in or absent from specific calculator categories, and the stratified subset of vignettes where every condition invoked tools.

4.6 Cost and Latency

Figure 6 plots the cost–accuracy frontier; table 3 summarizes per-vignette token and latency numbers.

Table 3: Per-vignette tokens (from the full $N = 1066$ run) and median single-request latency (from a fixed-concurrency timing pass at one worker, $N = 50$). "API" tokens are Sonnet calls (extractor + post-hoc verifier in D); "On-GPU" tokens are Qwen3 via vLLM. Tokens are load-independent; latency is reported only from the fixed-concurrency pass because the full run uses a different worker count per condition (section 3.2) and its wall-clock is therefore not comparable across conditions.

Condition	Total tokens	API tokens	Latency (s)
A	[TBD]	[TBD]	[TBD]
B (baseline)	[TBD]	[TBD]	[TBD]
B′	[TBD]	[TBD]	[TBD]
B′+E (upfront)	[TBD]	[TBD]	[TBD]
B′+E (reactive)	[TBD]	[TBD]	[TBD]
B′+E (reactive + citations)	[TBD]	[TBD]	[TBD]
B′+E (reactive + k-shot)	[TBD]	[TBD]	[TBD]
C	[TBD]	[TBD]	[TBD]
D	[TBD]	[TBD]	[TBD]

[TBD] Compare per-vignette token cost and single-request latency across the four extractor pipelines. Expected ordering on total tokens: reactive < reactive+citations < reactive+k-shot < upfront (the upfront ~50K-token call dominates everything the reactive variants do). Expected ordering on latency: reactive ≈ upfront < reactive+k-shot, because k-shot dispatches its three samples sequentially (~3× the single-sample extraction latency, section 3.7.4). Latency is measured at fixed single-worker concurrency so the comparison reflects single-request deployment time rather than batch contention.

[FIGURE TBD]
Per-vignette token cost decomposed into primary-model (Qwen3) and external API (Sonnet) tokens, by condition.

Figure 5: Per-vignette token cost decomposed into primary-model and external API tokens.

[FIGURE TBD]
Cost–accuracy frontier across the 9 conditions.

Figure 6: Cost–accuracy frontier.

[FIGURE TBD]
Latency vs accuracy (left) and latency vs token cost (right) for the 9 conditions.

Figure 7: Latency vs accuracy (left) and latency vs token cost (right).

Table 4: Per-condition accuracy with Wilson 95% CIs ($N = 1066$). Primary-family contrasts marked with † are Bonferroni-corrected at family $\alpha = 0.05$.

Condition	Group	Accuracy	95% CI
A	anchor	[TBD]	[TBD]
B (baseline)	anchor	[TBD]	[TBD]
B′	anchor	[TBD]	[TBD]
B′+E (upfront)†	primary	[TBD]	[TBD]
B′+E (reactive)†	primary	[TBD]	[TBD]
B′+E (reactive + citations)†	primary	[TBD]	[TBD]
B′+E (reactive + k-shot)†	primary	[TBD]	[TBD]
C	exploratory	[TBD]	[TBD]
D	exploratory	[TBD]	[TBD]

Table 5: Pre-registered McNemar contrasts (Bonferroni $\alpha = 0.00833$). Discordant pairs (b, c): b = correct in left condition only; c = correct in right condition only.

Comparison	b	c	$p_{\text{Bonferroni}}$	Sig.?
B vs A	[TBD]	[TBD]	[TBD]	[TBD]
B′ vs B	[TBD]	[TBD]	[TBD]	[TBD]
B′+E (upfront) vs B′	[TBD]	[TBD]	[TBD]	[TBD]
B′+E (reactive) vs B′	[TBD]	[TBD]	[TBD]	[TBD]
B′+E (reactive + citations) vs B′+E (reactive)	[TBD]	[TBD]	[TBD]	[TBD]
B′+E (reactive + k-shot) vs B′+E (reactive)	[TBD]	[TBD]	[TBD]	[TBD]

Table 6: Secondary contrasts (uncorrected). Within-architecture placement comparison and exploratory verifier-mechanism comparators.

Comparison	b	c	p	Δ acc
B′+E (upfront) vs B′+E (reactive)	[TBD]	[TBD]	[TBD]	[TBD]
C vs B	[TBD]	[TBD]	[TBD]	[TBD]
D vs B	[TBD]	[TBD]	[TBD]	[TBD]

4.7 Dropped Vignettes and Completion

[TBD] Per-condition counts of vignettes dropped due to incomplete runs (API failure, malformed tool call exhaustion, re-prompt cap reached mid-stream). Per the pre-registered failure policy (table 2), dropped vignettes are excluded from accuracy analysis. If dropped-N exceeds 2% in any condition, investigation is required before interpretation.

5 Discussion

[DISCUSSION — principal-findings numbers TBD after rerun. The interpretive framework (sections 5.2 and 5.3), threats (section 5.5), and scope statements (section 5.7) are results-agnostic and final.]

5.1 Principal Findings

We restate the four pre-registered hypotheses (section 1.3) and the outcome of each, drawing only on the contrasts in sections 4.2 and 4.3.

- **H1/H2 (does best-practice verifier-guided extraction beat B′?). [TBD]** — report the McNemar verdict for upfront and reactive against B′, and state plainly whether either cleared the Bonferroni threshold.
- **H3 (citations). [TBD]** — whether citation grounding moved accuracy over the reactive baseline.
- **H4 (k-shot). [TBD]** — whether $k = 3$ majority voting moved accuracy over the reactive baseline.
- **Anchor (does tool access help?). [TBD]** — the B-vs-A and B′-vs-B contrasts, which bound the room any verifier has to improve and are reported here only as context, not as a claim about verification.

The interpretation in the remainder of this section applies whichever way the primary contrasts resolve; sections 5.2 and 5.3 are the lens we use to explain any residual gap between the best-practice extractor pipelines and B′, and they do not presuppose a particular sign of that gap.

5.2 Interpreting a Residual Gap: The Reference Layer

The verifier in this study is deterministic (section 3.6): exact match on enums, numeric tolerance on numbers, canonical-form match on booleans, and no flag at all when the extractor abstains. None of that machinery can synthesize a wrong flag on its own — a spurious flag requires a wrong reference, and the reference comes from the LLM extractor. The extracted reference is therefore the load-bearing component, and any gap between a best-practice extractor pipeline and B′ localizes there rather than in the comparison logic.

We state the limit as a cost model. Where the verifier's reference is produced by an extractor M_e over a required-field set $\mathcal{F}$, the verifier's net effect on accuracy is governed by the joint distribution of three terms: (a) the primary model's input-error rate on $\mathcal{F}$ — the errors a true flag can fix; (b) M_e's extraction-error rate on $\mathcal{F}$ — the errors that produce false flags; (c) the asymmetric cost of a false flag (which re-prompts and can flip a correct call to wrong) versus a true flag (which can fix a wrong call). Verification helps only

when true flags outweigh false flags under this cost. The schema gate (section 3.6) restricts the comparison to required fields, which shrinks the surface on which term (b) can act; the four extractor pipelines vary placement, output format, and sampling, each of which is a different attempt to lower term (b). **[TBD: tie the observed per-pipeline gaps to which term moved — e.g., whether reactive scoping or k-shot voting measurably lowered the false-flag rate.]**

A subtlety the cost model alone misses: "extraction error" is broader than "the extractor read the wrong value." A flag can fire when the extractor's value is semantically correct but serialized differently from the model's (the `sex=female` vs `sex=1` pattern traced in appendix A.5). The binding constraint there is the join of (extractor canonical form, verifier comparison strictness, model serialization) — any one of the three can drive a flag the other two cannot resolve. **[TBD: report whether canonical-form mismatches remain a meaningful share of required-field flags under schema-gating, per section 4.4.]**

5.3 Why Tool-Forcing Amplifies the Reference

The verifier fires once per tool call, so its leverage scales with the tool-call rate. The B′ prompt forces a tool call on every vignette and removes the model's option to decline when uncertain; on every such call the verifier injects feedback carrying the extracted reference. This amplifies whatever the reference does: a reliable reference is applied more often (more fixes), and an unreliable one is applied more often (more mis-corrections). The amplification is multiplicative in the tool-call rate, and its sign is set entirely by the reference quality of section 5.2 — forcing is not harmful or helpful on its own. **[TBD: report the per-condition firing rate and intervention rate from section 4.4 and state which direction the amplification ran.]**

5.4 Implications for Deploying Verifier-Guided Reasoning in Clinical AI

Two recommendations follow from the mechanism regardless of the headline numbers; a third is conditional on them.

Validate the reference pipeline independently before adopting intermediate verification. The verifier's correctness guarantee is no stronger than the extractor's field-level accuracy on the deployment's live field distribution. A held-out, hand-annotated field-accuracy gate on the fields the deployment actually uses — not the union of all possible MCP fields — is the precondition the cost model in section 5.2 implies.

Prefer a non-LLM reference where one exists. Where structured patient data are available (EHR fields, device vitals, LIS results), they remove term (b) for the fields they carry. Reserve LLM extraction for fields the structured record does not hold.

Choose the extractor pipeline on the measured cost–accuracy frontier. **[TBD: once section 4.6 is in, state which pipeline is Pareto-efficient on tokens and latency and under what accuracy parity.]**

5.5 Threats to Validity

Single primary model. All conditions evaluate Qwen3-30B-A3B-Thinking-2507. The mechanism we identify (the extracted reference as the binding component) should transfer across models, but the magnitudes will not.

Single benchmark. medical_calculator_eval covers deterministic calculator dispatch with typed inputs.

Generalization to differential-diagnosis, triage, or open-ended tool surfaces — where inputs are less structured — is untested.

Single extractor model. Sonnet 4.6 is one extractor; another could show different per-field error rates and thus a different verdict. The claim is that *some* LLM extractor is the binding constraint, not that Sonnet specifically fails.

No ground-truth-reference comparator. No condition consults a hand-annotated reference, which would isolate the verifier mechanism from extractor fallibility entirely. This is the most important comparator the study lacks (section 5.6).

Conservative verifier is silent on extractor misses. The "no extractor value → no flag" rule (section 3.6) suppresses false positives but makes a silent extractor (one that extracts nothing for a field the model proposes) invisible to the firing-rate statistics. Some fraction of vignettes may carry silent verifier failures the firing counts cannot see.

Extractor stochasticity. The extractor runs at temperature 0.7 (and $k = 3$ in one pipeline), so its reference is not deterministic across runs. Per-field *value* agreement across repeated extractions is high; *field-presence* agreement is lower, and because of the conservative no-flag rule, presence variance translates into missing flags rather than wrong flags. A repeated run would likely shift total flag counts slightly without changing the architectural pattern. **[TBD: confirm against the rerun's reproducibility check, section 4.4.]**

Apparatus-gate scope. The gate (section 3.9) detects drift on the harness, dataset, prompt rendering, and MCP transport via Sonnet on Condition B. It does not independently validate the primary Qwen3 numbers; we assume the harness behaves identically for the primary model conditional on the gate holding.

5.6 Future Directions

The study fixes the verifier and varies only the extractor pipeline. Three reference-layer variants are the natural next experiments, each holding all other variables at this study's values. We list them as hypothesized future work, not as findings.

- **Hand-annotated ground-truth reference.** Removes the LLM extractor entirely and answers whether procedural input verification helps when the reference is reliable. Cost: annotation of all 1,066 vignettes.
- **Multi-LLM ensemble extractor.** Requires agreement across ≥ 2 extractor models before the verifier's reference is populated, targeting single-model variance in term (b).
- **Structured-record reference.** A non-LLM reference layer (e.g., EHR-keyed inputs) for a deployment-realistic setting; requires a benchmark that ships structured records alongside vignettes, which medical_calculator_eval does not.

5.7 What This Paper Does Not Claim

To preempt common misreadings, we name the claims this paper does *not* make.

1. "Interwhen does not work." Interwhen's verify/fix mechanism operates as designed in every condition we run. Our adaptation repurposes its LLM extraction step to recover an independent reference from

the case rather than the model's asserted state from the trace (section 2.1); any limit we find is in that reference layer, not the mechanism.

2. "Output verification is unhelpful." We study procedural *input* verification specifically (section 2.3). Output verification, outcome verification, grounded retrieval, and human-in-the-loop regimes are different surfaces; Condition D is the only output-style comparator here and is scoped as a control, not the object of study.

3. "Sonnet is a poor extractor." Sonnet 4.6 is near the frontier of clinical-text extraction. Any underperformance we attribute to the extractor is a property of the prompt regime (e.g., a ~500-field upfront schema), not the model's underlying capability.

4. "Tool-forcing prompts are dangerous." Forcing amplifies whatever follows it in both directions (section 5.3). With a reliable reference the amplification is beneficial; the prompt is not harmful on its own.

5. "Any one pipeline is the right deployment configuration." Each contrast is one mechanistic test. A production choice depends on the measured cost–latency frontier, field-level validation of the extractor, and the specific clinical workflow — more than a single accuracy contrast can settle.

6 Conclusion

[CONCLUSION TBD — update after primary study rerun.]

We evaluated interwhen-style procedural input verification on a clinical tool-calling benchmark across nine conditions spanning three groups: capability anchors (A, B, B′), a primary study of four extractor pipelines under best-practice configuration (upfront, reactive, reactive + citations, reactive + k-shot), and two exploratory verifier-mechanism comparators (C, D). Six paired contrasts were pre-registered at family $\alpha = 0.05$.

[TBD] Replace this paragraph with the bottom-line conclusion after the rerun: (1) whether best-practice verifier-guided extraction moved accuracy above B′; (2) which architectural axis carried the effect, if any; (3) the resulting deployment guidance.

Two design choices in the primary study — the schema-gated verifier and the query-style intervention — were motivated by pilot diagnostics (section 3.5) and address by construction two failure modes those pilots exposed. The rerun quantifies the residual gap, if any, between the best-practice extractor and the no-verifier forced-tool baseline.

Code and Data Availability

The full experimental harness — including the orchestration notebook, per-condition adapter code, the deterministic input verifier, the schema-aligned fact extractor, and the analysis pipeline — is available at `https://github.com/jackyang25/interwhen-karma-harness`. The *medical_calculator_eval* benchmark is distributed by EkaCare via the KARMA framework; we use the public release without modification.

The complete per-condition result bundle (raw model outputs, scored rows, McNemar contingency tables, manifests with SHA-256 checksums, the MCP calculator schema dump used at runtime, and the regenerated extractor prompt) is archived at [ARCHIVE_DOI_OR_PATH].[1] Schema dumps are included in each bundle's provenance subdirectory so the extractor prompt can be regenerated byte-identically from the source schemas.

All runs are deterministic by construction: model temperature is zero, the schema dump is frozen at run start, and the extractor prompt is generated deterministically from the dump. Re-running the harness on the same benchmark with the same seeds and the same MCP schema dump reproduces the per-vignette outcomes used in this paper.

Funding and Conflicts of Interest

This work was conducted as part of the authors' employment with the Bill & Melinda Gates Foundation. No external grants supported this study. The authors declare no competing financial or non-financial interests related to this work.

Acknowledgments

The authors thank the interwhen team at Microsoft Research India for discussions of the verifier-guided reasoning framework this work adapts, and for making the reference implementation available. We thank the EkaCare team for releasing the *medical_calculator_eval* benchmark and the MedAI MCP toolset that made this study possible. The Qwen team is acknowledged for the public release of Qwen3-30B-A3B-Thinking-2507, and Anthropic for the Sonnet 4.6 model used as the fact extractor.

References

[1] Anthropic. Claude sonnet 4.6. Anthropic model release, 2026. URL https://www.anthropic.com/claude.

[2] V. K. Bhat, P. Chanda, V. Ekbote, A. Khandelwal, M. Swaroop, V. N. Balasubramanian, S. Kambhampati, N. Natarajan, and A. Sharma. interwhen: A generalizable framework for steering reasoning models with test-time verification. *arXiv preprint arXiv:2602.11202*, 2026. URL https://arxiv.org/abs/2602.11202.

[3] EkaCare. Beyond frontier models: Grounding ai in indian clinical reality. EkaCare blog, 2024. URL https://www.eka.care/services/beyond-frontier-models-grounding-ai-in-indian-clinical-reality.

[4] EkaCare. Introducing karma / openmedevalkit: An open-source framework for medical ai evaluation. EkaCare blog, 2024. URL https://www.eka.care/services/introducing-karma-openmedevalkit-an-open-source-framework-for-medical-ai-evaluation.

[1] **[USER INPUT NEEDED]** Replace with the Zenodo/Figshare/Drive location of the archived bundle once published. Each bundle ships with a manifest JSON that indexes every artifact and a SHA-256 checksum.

[5] EkaCare. Medai: Model context protocol tools for clinical calculators. Distributed with the KARMA framework, 2024. URL https://www.eka.care/services/introducing-karma-openmedevalkit-an-open-source-framework-for-medical-ai-evaluation.

[6] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165.

[7] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[8] Qwen Team. Qwen3-30b-a3b-thinking-2507. Hugging Face model card, 2025. URL https://huggingface.co/Qwen/Qwen3-30B-A3B-Thinking-2507.

[9] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

A Reference Artifacts

A.1 System Prompts

The system prompts below differentiate conditions B′, C, and D. The four primary study conditions (the B′+E pipelines) all use the B′ prompt (listing 1). Conditions A and B use no system prompt (the dataset's per-row *confinement instruction*, described in section 3.1, is the only prompt suffix appended).

```
You are an expert clinician answering medical calculator questions for
Indian patients.

You must call the appropriate calculator tool for every computation — do
not compute values by hand, even if you believe the answer is obvious.
Always invoke the matching `medical_calculator_*` tool with inputs taken
directly from the patient case, and use the tool's returned value as
your answer.

Read the case carefully, and after the tool returns, reply strictly
with the JSON object the user requests.
```

Listing 1: Condition B′ — force tool use. Verbatim from *prompts/condition_b_prime.txt*.

```
You are an expert clinician answering medical calculator questions for
Indian patients.

Before calling any calculator tool, verify each input you plan to pass
against the patient case: confirm the units match (e.g. mg/dL vs
mmol/L), the enum choices are correct (e.g. sex, activity level,
severity), and the numeric values are taken directly from the vignette.
If the case lacks clear evidence for a required input, do not guess —
re-read the case before calling the tool.

Read the case carefully, use any available tools if useful, and reply
strictly with the JSON object the user requests.
```

Listing 2: Condition C — prompt-only self-verify. Verbatim from *prompts/condition_c.txt*.

```
You are a verification reviewer for clinical calculator answers.

You will be shown (1) a patient case in Indian clinical language, (2)
the question's required output format, and (3) a candidate JSON answer
produced by another model. Your job is to identify whether the
candidate answer is consistent with the case — specifically, whether
the inputs the answering model used (if visible in its reasoning) match
what the case states.

Reply with a JSON object of the form:
{
```

```
  "consistent": true | false,
  "issue": "<one-sentence description of the discrepancy if
   consistent=false, otherwise empty string>"
}

Be conservative: only flag clear inconsistencies in units, enum choices
(sex, activity level, severity, etc.), or numeric values that
contradict the case. Do not flag minor formatting differences or
stylistic choices. Do not flag a missing field unless the question
requires it. If you are not certain there is an inconsistency, return
consistent=true.
```

Listing 3: Condition D — post-hoc Sonnet verifier system prompt. Verbatim from *prompts/condition_d.txt*.

A.2 Fact Extractor Prompt

The fact extractor used in all four primary study (B′+E) conditions runs Sonnet with the system prompt shown in listing 4, followed by a schema body that is generated deterministically from the MCP per-calculator JSON Schemas. For the upfront variant the body is the full union of all calculator fields (~500 entries); for the reactive variant it is the subset of fields the active tool call requests (typically 3–30). The header and rules are byte-identical across the two variants; only the schema body differs.

```
You are a medical fact extractor. Read the patient vignette and return
a strict JSON object capturing every value the case explicitly states
for the fields below.

Return ONLY a JSON object (no prose, no markdown fences). Include ONLY
the fields the case explicitly states a value for. Omit any field the
case does not state — do not emit null for absent fields. The schema
below is the vocabulary of allowed field names; you do not need to
emit every field.

Rules:
- Include only what the case explicitly states. If a value must be
  inferred, computed, or guessed, omit the field.
- Use the units the case provides. Do not convert units. If the case
  says "creatinine 130 μmol/L", store 130, not the converted mg/dL value.
- For ranges (e.g. "BP 140/90"), split into the appropriate fields.
- For descriptors that map to a calculator's enum (e.g. "sedentary
  lifestyle" →"sedentary"), use the enum value if the mapping is
  unambiguous; otherwise omit.
- If the case uses Hinglish or non-standard phrasing, interpret in
  standard clinical context where unambiguous; otherwise omit.
- Return only valid JSON. No commentary.

Schema (allowed field names with type/range/enum constraints):
{ ...generated schema body... }
```

Listing 4: Fact extractor system prompt header (stable across upfront and reactive variants). The schema body that follows is generated programmatically from the MCP schema dump by *harness/extraction/prompt_builder.py*.

A.3 Verifier Feedback Templates

On a flag, the verifier injects a feedback template into the primary model's prompt. The *{violations}* placeholder expands to a list of disagreeing fields, each annotated with the field name, the model's planned value, and the extractor's value. The primary study uses the *query-style* template (listing 5); the *correction-style* template (listing 6) was used only in the pilot diagnostics (section 3.5) and is reproduced here for contrast.

```
A second reading of the patient case (by an independent extractor)
disagrees with one or more of the input values you planned to pass to
this tool call:

{violations}

Please re-read the case carefully and determine, for each disagreeing
field, which value the vignette actually supports. If the case clearly
supports your original value, keep it. If the case clearly supports the
extractor's value, use that. If the case does not clearly specify the
field, return that field's value as null and explain your reasoning.

Then call the tool again with the values the case supports.
```

Listing 5: Query-style intervention template used in all primary study conditions. Verbatim from *prompts/condition_e_feedback_query.txt*. Per section 3.4, it presents the disagreement as a question to be resolved against the vignette, without asserting that either side is correct.

```
Your planned tool call has an input that does not match the patient case:

{violations}

Re-read the case carefully and call the tool again with corrected
inputs. If the case truly does not specify the required input, do not
invent a value; explain in your reply.
```

Listing 6: Correction-style template (pilot only, not used in the primary study). Verbatim from *prompts/condition_e_feedback.txt*.

A.4 Example MCP Calculator Schema

Across all ~70 calculators in MedAI MCP, the union of property names exceeds 500 unique fields; the schema for any single calculator typically declares 3–30. One representative calculator's JSON Schema is reproduced verbatim below.

```
{
  "name": "bmi_calculator_body_mass_index",
  "input_data": {
    "type": "object",
    "required": ["height", "mass"],
    "properties": {
      "sex":     { "$ref": "#/$defs/Sex", "description": "Biological sex" },
      "height":  { "type": "number",  "description": "Height in meters" },
      "mass":    { "type": "number",  "description": "Body weight in kilograms
          " },
      "age":     { "type": "integer", "description": "Age in years" },
      "athlete": { "$ref": "#/$defs/YesNo", "description": "Athletic status" }
    }
  }
}
```

Listing 7: Abbreviated MCP JSON Schema for *bmi_calculator_body_mass_index*. The full schema includes descriptions, units, and range constraints for each field. Note that `sex` is declared but *not* required; the BMI calculator will dispatch without it.

A.5 Real-Data Trace: Vignette `bmi_001`

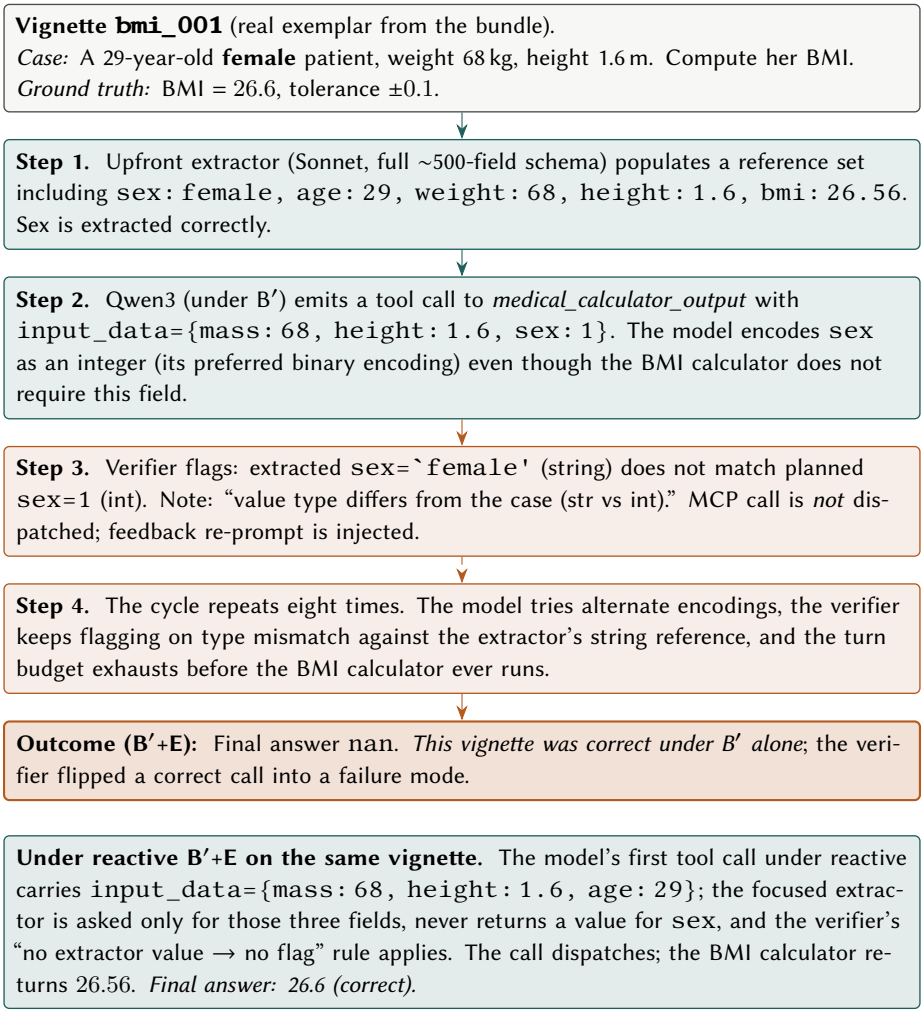

Figure 8: Trace of vignette `bmi_001`: an unrequested `sex` value in the upfront reference triggers a spurious flag-loop that reactive extraction avoids.

Figure 8 walks through one real exemplar from the pilot diagnostic runs described in section 3.5. Under the un-gated upfront B′+E pilot, the extractor populated a value for `sex` that the BMI calculator did not request; the model's tool call used a different serialization (`sex=1` vs the extracted `sex=`female'`), triggering an unresolvable type-mismatch loop that exhausted the turn budget. Under schema-gated comparison (section 3.6) this class of flag is suppressed by construction: `sex` is not declared as required by the BMI calculator's JSON Schema (listing 7), so the verifier skips the field and the calculator dispatches cleanly.

A.6 Implementation Methodology

This appendix describes how the pieces of the verification stack fit together at a methodology level. It is not a code reference; the canonical implementation lives in the harness repository under `harness/`.

A.6.1 Three Orthogonal Axes

Every extractor pipeline in the primary study is a point in a three-axis configuration space:

1. **Extraction placement.** *Upfront* runs one full-schema Sonnet call before the model begins generation; *reactive* runs one scoped Sonnet call at each *medical_calculator_output* tool-call interception.
2. **Output format.** *Bare* returns `{field: value}`; *citation* returns `{field: {value, source_span}}` where `source_span` must be a verbatim substring of the vignette text.
3. **Sampling.** *Single* dispatches one Sonnet call; *k-shot* dispatches $k = 3$ independent calls at temperature 0.7 and reduces per-field by majority vote.

The four primary study conditions are four points in this space; no condition mixes two non-default values on different axes. This preserves orthogonality for the pairwise contrasts in section 3.8.

A.6.2 Components

Five components participate in every primary study condition; their behavior is identical across conditions except where noted.

Primary model. Qwen3-30B-A3B-Thinking-2507 served by vLLM on the H100. Receives the case vignette plus a confinement instruction, emits a stream of generation that may contain one or more `<tool_call>` blocks before the final JSON answer. Identical configuration across all conditions.

Tool-call loop driver. Intercepts generation at `</tool_call>` stop tokens, dispatches the call to the verifier (in extractor-enabled conditions) or directly to MCP (in A/B/B′), splices the tool's structured response back into the prompt, and resumes generation. Identical across all conditions; the only inter-condition difference is whether the call passes through the verifier.

Extractor. Claude Sonnet 4.6 with a system prompt header (listing 4) and a schema body. The schema body differs across pipelines: the upfront pipeline uses the union of all calculator schemas; reactive pipelines use the subset of fields the active calculator declares. The citation variant additionally requires a `source_span` per populated field; the k-shot variant dispatches three samples in parallel at temperature 0.7 and votes per field. The extractor is blind to the model's proposed tool-call values — it sees only the case text and (in reactive variants) the names of the fields the active calculator requests.

Verifier. A deterministic comparator with three filters applied in order: (1) the *schema gate* restricts comparison to fields the active calculator declares as required; (2) the *abstention gate* skips fields the extractor returned as null; (3) the *deterministic comparator* applies exact-match, numeric-tolerance, or canonical-form rules and flags on mismatch. On flag, the verifier injects the query-style intervention prompt (appendix A.3) into the model's prompt and the loop driver suppresses the MCP dispatch for that turn. The re-prompt counter is incremented; when it reaches the cap in table 2, subsequent flags are demoted to warnings and the MCP call is allowed through.

Validator (citation variant only). A post-extraction filter that verifies each `source_span` is a byte-identical substring of the vignette text. Validation failures coerce the field to null, which the verifier's abstention gate then skips.

A.6.3 Per-Condition Adapter Behavior

Table 7 summarizes the adapter behavior for each primary study condition. "Adapter" here refers to the boundary between the loop driver and the extractor/verifier; the loop driver itself is identical across all conditions.

Table 7: Adapter behavior per primary study condition. Loop driver, primary model, MCP transport, and verifier comparison logic are byte-identical across all four; differences live entirely in the extractor pipeline.

Condition	Extractor trigger	Sonnet calls per vignette	Per-field aggregation
B′+E (upfront)	once, before first tool call	1 (full schema)	identity
B′+E (reactive)	at each *_output* call	∼3–4 (scoped)	identity
B′+E (reactive + citations)	at each *_output* call	∼3–4 (scoped, with citations)	span-validate; coerce to null on failure
B′+E (reactive + k-shot)	at each *_output* call	∼3–4 × k=3 (scoped)	majority of 3; no-majority → null

A.6.4 Pilot-Batch Sanity Checks

The five pre-run sanity checks from section 3.4 target the implementation invariants that would silently invalidate comparisons if violated:

1. **k-shot sample divergence.** For the k-shot condition, pull the three Sonnet responses on a small sample of vignettes and confirm they differ string-wise. Failure indicates prompt caching or zero-temperature sampling is collapsing samples.

2. **Qwen3 input parity across conditions.** Hash the prompt assembled for the primary model on a fixed vignette under conditions B and B′. The system-prompt differs by design; the case-text portion must be bit-identical. Failure indicates preprocessing drift.

3. **Re-prompt cap enforcement.** Construct a vignette that repeatedly triggers a verifier flag and confirm the loop halts at the cap rather than looping indefinitely.

4. **Abstention firing.** On a vignette with deliberately absent information, confirm the extractor returns `null` for the absent field rather than fabricating a value.

5. **Citation validator rejection.** Inject a paraphrased `source_span` (e.g., normalized units) and confirm the validator rejects it, coercing the field to null.

These checks are run on a 30-vignette pilot batch and must pass before the full $N = 1066$ run commences.

A.7 Metadata and Logging

All conditions produce a uniform per-vignette log structure so that downstream analysis (accuracy, McNemar contrasts, flag composition, cost decomposition) can be run identically against any condition's output.

A.7.1 Per-Run Manifest

Each condition's run produces a single manifest JSON capturing:

- **Identity.** Condition ID, condition group (anchor / primary / exploratory), pre-registration config hash.
- **Model snapshots.** Qwen3 weights hash, vLLM version, Sonnet snapshot identifier, MCP backend version.
- **Sampling configuration.** Qwen3 temperature, top-p, max-tokens; Sonnet extraction temperature, k samples per field, seed strategy.
- **Hygiene flags.** Schema-gating enabled (true/false), intervention style (`query` / `correction`), abstention mechanism (`prompt-only`).
- **Apparatus.** Hardware (H100 instance type), Databricks runtime, worker count, start/end timestamps.
- **Apparatus-gate result.** Sonnet-on-B replication accuracy and pass/fail (recorded for the gate run only, with a back-reference from each downstream condition's manifest).

The manifest is written before the first vignette is dispatched and finalized at run end; the pre-registration config hash is verified to match the locked config on disk.

A.7.2 Per-Vignette Log Record

Each vignette in each condition produces one JSON record. The schema is identical across conditions; fields that do not apply to a condition (e.g., extractor fields under condition A) are omitted, not filled with null.

- **Identity.** Vignette ID, condition ID, run ID.
- **Input.** Case text hash, confinement instruction hash, target calculator, expected value, tolerance.
- **Extractor calls** (extractor-enabled conditions only). Ordered list, one entry per Sonnet dispatch:
  - Trigger point (`upfront` or the *_output* call index).
  - Scope: list of field names the call was prompted for.
  - Sample index (for k-shot: $0, 1, 2$; otherwise 0).
  - Sonnet input tokens, output tokens, latency.
  - Raw response (string).
  - Parsed fields: `{field: value}` or `{field: {value, source_span}}` per output format.
  - Citation validation results (citation condition): per-field pass/fail and the reason on fail.
  - k-shot aggregation (k-shot condition): per-field votes and the majority outcome.

- **Tool calls.** Ordered list, one entry per `<tool_call>` the primary model emitted:
  - Turn index, calculator name, parsed input arguments.
  - Verifier decision (`pass` / `flag` / `cap-reached`).
  - On flag: the disagreeing fields, the model's value, the extractor's value, and the rendered intervention prompt.
  - Dispatched (true/false); MCP response if dispatched.
  - Per-call wall-clock: extractor dispatch time (sum across k samples for k-shot), verifier comparison time, MCP dispatch time. Enables decomposing end-to-end latency into extractor vs. tool-call vs. model-generation components.
- **Re-prompt counter.** Final value of the per-vignette re-prompt counter; flag indicating whether the cap was reached.
- **Final answer.** The model's terminal JSON response, the parsed numerical prediction, the correctness verdict.
- **Token accounting.** Per-vignette totals: Qwen3 input, Qwen3 output, Sonnet input, Sonnet output.
- **Wall-clock.** Per-vignette end-to-end elapsed seconds (a scalar, not a pre-aggregated median — the aggregation into per-condition medians happens at analysis time).
- **Completion status.** `ok`, `api-error`, `malformed-tool-call-exhausted`, `turn-budget-exhaus` Dropped iff status ≠ `ok`, per table 2.

A.7.3 Export Format

The per-vignette records are written as JSON Lines (`.jsonl`), one file per condition, named `<condition_id>__<run` JSON Lines is chosen for streamability (large files load incrementally), for compatibility with `pandas.read_json(lin` during analysis, and because nested structures (extractor calls, tool calls) live naturally in JSON. A companion `<condition_id>__<run_id>.manifest.json` captures the per-run metadata above. Both files are committed to the result bundle under `results/<run_id>/` and referenced by hash from any figure or table that reports their contents.

A.7.4 Analysis Surface

The accuracy, flag-composition, and cost tables in the results section are produced by uniform reduction functions over the `.jsonl` files. The same reduction function applied to two different conditions' files produces directly comparable outputs; no per-condition special-casing is permitted in analysis code. This property is the methodological reason for the schema-uniformity requirement above.